MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 17 — Gestion des Erreurs

try, except, raise — écrire du code robuste et professionnel

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

15 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 17 — Gestion des Erreurs **2**
0.1 Les types d'erreurs en Python 2
0.2 La hiérarchie des exceptions 2
0.3 try / except — Intercepter les erreurs 3
0.4 else et finally — Compléter la structure 3
0.5 Intercepter plusieurs exceptions 5
0.6 raise — Lever ses propres exceptions 6
0.7 Créer ses propres exceptions 6
0.8 Le gestionnaire de contexte — with 8
0.9 Les assertions — Vérifier les invariants 9
0.10 Ce qui m'a le plus surpris dans ce chapitre 10
0.11 Résumé du Chapitre 17 10

Chapitre 17 — Gestion des Erreurs

À retenir

Les erreurs sont inévitables dans tout programme. L'utilisateur saisit une mauvaise valeur, un fichier est introuvable, une connexion réseau échoue. Un programme professionnel ne **plante** pas — il **gère** ces situations avec élégance. Python offre un système d'exceptions puissant et expressif : `try`, `except`, `else`, `finally` et `raise`. Maîtriser ce système, c'est écrire du code **robuste, fiable et digne de production**.

Les types d'erreurs en Python

Python distingue deux grandes familles d'erreurs.

SyntaxError

Détectée AVANT l'exécution

```
if x = 5:
def f(
print "hello"
```

Le programme refuse de démarrer. À corriger avant tout.

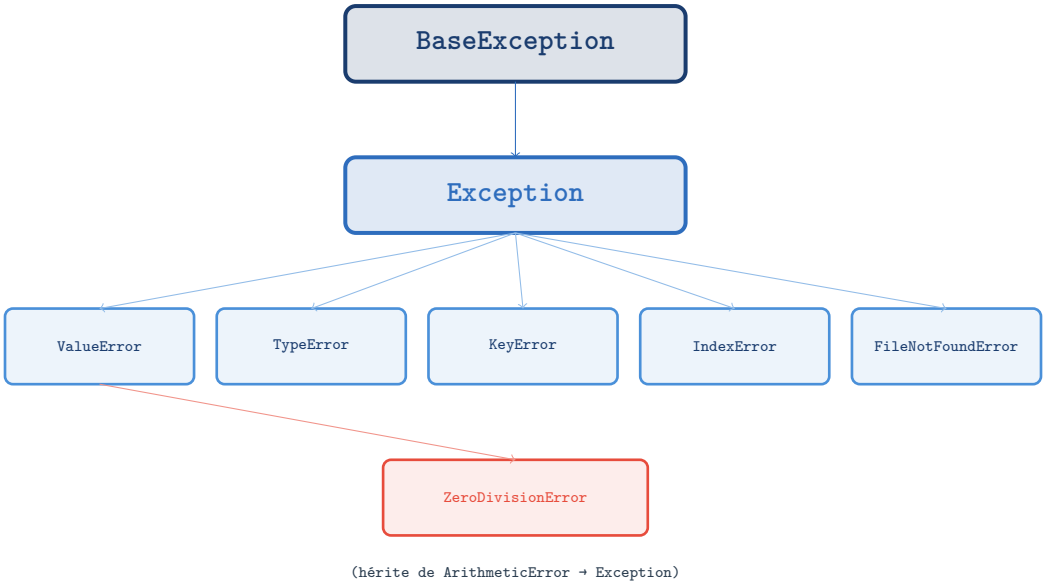
Exception (RuntimeError)

Détectée PENDANT l'exécution

```
1 / 0
liste[99]
int("abc")
```

Le programme démarre mais rencontre un problème. Peut être interceptée.

La hiérarchie des exceptions



try / except — Intercepter les erreurs

Structure try / except de base

```python
# Sans gestion d'erreur      le programme plante
# resultat = 10 / 0        ZeroDivisionError !

# Avec try / except      le programme continue
try:
    resultat = 10 / 0
except ZeroDivisionError:
    print("Erreur : division par zero !")
    resultat = 0

print(f"Resultat : {resultat}")    # Resultat : 0

# Capturer le message d'erreur avec 'as'
try:
    nombre = int("abc")
except ValueError as e:
    print(f"Erreur de valeur : {e}")
    #      invalid literal for int() with base 10: 'abc'

# Intercepter plusieurs types d'erreurs
def diviser(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Division par zero impossible.")
        return None
    except TypeError as e:
        print(f"Type incorrect : {e}")
        return None

print(diviser(10, 2))      # 5.0
print(diviser(10, 0))      # None + message
print(diviser(10, "a"))    # None + message
```

else et finally — Compléter la structure

try / except / else / finally

```python
# else       s'execute si AUCUNE exception n'a ete levee
# finally      s'execute TOUJOURS (avec ou sans exception)

def lire_nombre(chaine):
    try:
        nombre = int(chaine)
    except ValueError:
```

```python
        print(f"      '{chaine}' n'est pas un nombre entier.")
        return None
    else:
        # Execute seulement si try a reussi
        print(f"      Conversion r ussie : {nombre}")
        return nombre
    finally:
        # Execute TOUJOURS     nettoyage, fermeture de ressources
        print(f"      Tentative terminee pour '{chaine}'")

print("Test 1 :")
lire_nombre("42")

print("\nTest 2 :")
lire_nombre("abc")

print("\nTest 3 :")
lire_nombre("3.14")
```

finally — libérer les ressources

```python
def traiter_fichier(nom):
    fichier = None
    try:
        fichier = open(nom, "r")
        contenu = fichier.read()
        print(f"Lu {len(contenu)} caracteres.")
        return contenu
    except FileNotFoundError:
        print(f"Fichier '{nom}' introuvable.")
        return None
    except PermissionError:
        print(f"Permission refusee pour '{nom}'.")
        return None
    finally:
        # Le fichier est TOUJOURS ferme     meme en cas d'erreur
        if fichier:
            fichier.close()
            print("Fichier ferme proprement.")

traiter_fichier("inexistant.txt")
```

À retenir

`finally` est garanti de s'exécuter **dans tous les cas** — succès, exception interceptée, ou même `return` dans le `try`. C'est l'endroit idéal pour libérer des ressources : fermer des fichiers, des connexions de base de données, des sockets réseau. Ne jamais omettre ce nettoyage dans du code de production.

Intercepter plusieurs exceptions

Gérer plusieurs exceptions

```python
def convertir_et_diviser(a_str, b_str):
    try:
        a = int(a_str)
        b = int(b_str)
        resultat = a / b
        return resultat

    except ValueError as e:
        print(f"Valeur invalide : {e}")
    except ZeroDivisionError:
        print("Division par zero !")
    except (TypeError, AttributeError) as e:
        # Capturer plusieurs types dans un seul except
        print(f"Erreur de type : {e}")
    except Exception as e:
        # Filet de securite     attrape tout le reste
        print(f"Erreur inattendue ({type(e).__name__}) : {e}")
    finally:
        print("Operation terminee.\n")

convertir_et_diviser("10", "2")
convertir_et_diviser("10", "0")
convertir_et_diviser("abc", "2")
convertir_et_diviser("10", "abc")
```

Erreur fréquente

Ne jamais écrire `except Exception` seul sans logger l'erreur. Avaler silencieusement toutes les exceptions masque des bugs réels. Le `except Exception` doit toujours être un **dernier recours** avec un message explicite, et placé après tous les `except` spécifiques. L'ordre des `except` compte : du plus spécifique au plus général.

raise — Lever ses propres exceptions

raise — signaler une erreur volontairement

```python
def calculer_racine(nombre):
    if not isinstance(nombre, (int, float)):
        raise TypeError(f"Attendu int ou float, recu {type(
            nombre).__name__}")
    if nombre < 0:
        raise ValueError(f"Impossible de calculer la racine de
            {nombre}")
    import math
    return math.sqrt(nombre)

# Test
for valeur in [25, -4, "abc", 0]:
    try:
        resultat = calculer_racine(valeur)
        print(f"   {valeur} = {resultat}")
    except ValueError as e:
        print(f"ValueError : {e}")
    except TypeError as e:
        print(f"TypeError  : {e}")

# Re-lever une exception apres traitement partiel
def charger_config(fichier):
    try:
        with open(fichier) as f:
            return f.read()
    except FileNotFoundError as e:
        print(f"Config manquante, utilisation des defauts.")
        raise   # Re-leve l'exception originale telle quelle

# Lever depuis une autre exception (chainage)
def connecter_base(url):
    try:
        raise ConnectionError("Timeout reseau")
    except ConnectionError as e:
        raise RuntimeError("Impossible de demarrer l'
            application") from e
```

Créer ses propres exceptions

Exceptions personnalisées

```python
# Toutes les exceptions heritent de Exception
class ErreurSoldeInsuffisant(Exception):
    """Levee quand le solde est insuffisant pour un retrait."""
```

```python
    def __init__(self, solde, montant):
        self.solde   = solde
        self.montant = montant
        super().__init__(
            f"Solde insuffisant : {solde}    disponibles, "
            f"{montant}    demand s (manque {montant - solde}
                )"
        )


class ErreurCompteBloque(Exception):
    """Levee quand un compte est bloque."""

    def __init__(self, raison="Raison inconnue"):
        self.raison = raison
        super().__init__(f"Compte bloque : {raison}")


class CompteBancaire:
    def __init__(self, titulaire, solde=0):
        self.titulaire = titulaire
        self._solde    = solde
        self._bloque   = False

    def bloquer(self, raison):
        self._bloque = True
        self._raison = raison

    def retirer(self, montant):
        if self._bloque:
            raise ErreurCompteBloque(self._raison)
        if montant > self._solde:
            raise ErreurSoldeInsuffisant(self._solde, montant)
        self._solde -= montant
        return self._solde


# Utilisation
compte = CompteBancaire("Alice", 500)

try:
    compte.retirer(200)
    print("Retrait OK")
    compte.retirer(400)    # Solde insuffisant !
except ErreurSoldeInsuffisant as e:
    print(f"REFUS : {e}")
    print(f"  Solde   : {e.solde}   ")
    print(f"  Demande : {e.montant}   ")

compte2 = CompteBancaire("Bob", 1000)
```

```python
compte2.bloquer("Activite suspecte d tectee")

try:
    compte2.retirer(100)
except ErreurCompteBloque as e:
    print(f"BLOQU  : {e}")
    print(f"  Raison : {e.raison}")
```

Le gestionnaire de contexte — with

with — gestion automatique des ressources

```python
# Sans with     il faut gerer manuellement
fichier = open("notes.txt", "w")
try:
    fichier.write("Bonjour Python\n")
    fichier.write("Chapitre 17\n")
finally:
    fichier.close()   # toujours fermer !

# Avec with     fermeture AUTOMATIQUE et garantie
with open("notes.txt", "w") as fichier:
    fichier.write("Bonjour Python\n")
    fichier.write("Chapitre 17\n")
# fichier est ferme automatiquement ici     meme si exception

# Lire un fichier proprement
try:
    with open("notes.txt", "r") as f:
        for ligne in f:
            print(ligne.strip())
except FileNotFoundError:
    print("Fichier introuvable.")

# Plusieurs gestionnaires en une seule ligne (Python 3.10+)
with open("source.txt") as src, open("dest.txt", "w") as dst:
    dst.write(src.read())

# Creer son propre gestionnaire de contexte
from contextlib import contextmanager

@contextmanager
def chronometre(nom="Operation"):
    import time
    print(f"    {nom} demarre...")
    debut = time.time()
    try:
        yield   # le bloc with s'execute ici
```

```
    finally:
        duree = time.time() - debut
        print(f"    {nom} termin e en {duree:.4f}s")

with chronometre("Calcul intensif"):
    total = sum(i**2 for i in range(1_000_000))
    print(f"  Resultat : {total}")
```

Ce qui m'a résisté

Le gestionnaire de contexte `with` m'a semblé accessoire au début — une syntaxe de confort. J'ai réalisé son importance quand j'ai eu une exception à l'intérieur d'un bloc `with` : le fichier s'est quand même fermé proprement, alors qu'avec un `try/finally` manuel j'aurais pu oublier le `close()`. `with` n'est pas du sucre syntaxique — c'est un **protocole** (les méthodes `__enter__` et `__exit__`) qui garantit la libération des ressources dans tous les cas.

Les assertions — Vérifier les invariants

assert — vérifications de développement

```
def calculer_moyenne(notes):
    # Verifications des preconditions
    assert isinstance(notes, list), "notes doit etre une liste"
    assert len(notes) > 0,          "La liste ne peut pas etre
       vide"
    assert all(0 <= n <= 20 for n in notes), \
           "Toutes les notes doivent etre entre 0 et 20"

    return sum(notes) / len(notes)

# OK
print(calculer_moyenne([15, 18, 12]))   # 15.0

# AssertionError avec message
try:
    calculer_moyenne([])
except AssertionError as e:
    print(f"AssertionError : {e}")

try:
    calculer_moyenne([15, 25, 12])   # 25 > 20 !
except AssertionError as e:
    print(f"AssertionError : {e}")
```

Note importante

Les `assert` sont désactivés quand Python est lancé avec l'option `-O` (optimisation). Ils sont faits pour le **développement**, pas pour la gestion d'erreurs en production. Pour valider des entrées utilisateur ou des données de production, toujours utiliser `raise` avec une exception appropriée.

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `else` dans un `try` — une sémantique inattendue. Comme le `else` des boucles (chapitre 6), le `else` du `try` m'a surpris. Il ne s'exécute pas "sinon il y a une erreur" — il s'exécute "si aucune erreur ne s'est produite". Cette distinction est subtile mais importante : elle sépare clairement le code qui peut échouer (`try`) du code qui ne doit s'exécuter qu'en cas de succès (`else`), améliorant la lisibilité.

2. `with` implémente un protocole — pas juste du sucre syntaxique. Découvrir que `with` repose sur `__enter__` et `__exit__` m'a révélé que Python est cohérent jusqu'au bout : tout ce qui semble "magique" est un protocole bien défini. Et créer son propre gestionnaire de contexte avec `@contextmanager` m'a montré qu'on peut rendre n'importe quelle ressource — pas juste les fichiers — safe par construction.

3. Les exceptions personnalisées portent des données. Au lieu d'un simple message de texte, une exception peut transporter des **attributs structurés** — le solde disponible, le montant demandé, la raison du blocage. L'appelant peut alors accéder à `e.solde` et `e.montant` pour prendre des décisions programmatiques. C'est une API d'erreur — aussi important à bien concevoir que l'API principale.

Résumé du Chapitre 17

Ce que j'ai appris dans ce chapitre :

- `try / except` — intercepter les erreurs sans planter le programme.
- `except TypeErreur as e` — capturer le message et les détails.
- `else` — s'exécute uniquement si **aucune** exception n'a été levée.
- `finally` — s'exécute **toujours**, nettoyage garanti.
- `raise` — lever une exception volontairement avec message.
- **Exceptions personnalisées** — hériter d'`Exception` avec attributs.
- `with` — gestionnaire de contexte, libération automatique des ressources.
- `assert` — vérifications de développement, désactivables en production.
- Hiérarchie des exceptions — du plus spécifique au plus général.

"Un programme robuste n'est pas celui qui ne rencontre jamais d'erreurs. C'est celui qui les anticipe, les gère, et continue à fonctionner."